

How Uber built an AI software factory for agentic coding: the MCP gateway and the platform underneath

More than 70% of pull requests at Uber now come from local or cloud agents, and lines of code per engineer has doubled year over year. Uday Kiran Medisetty shows six pieces of the AI Software Factory.



ZOHAR EINY
AUG 25, 2026



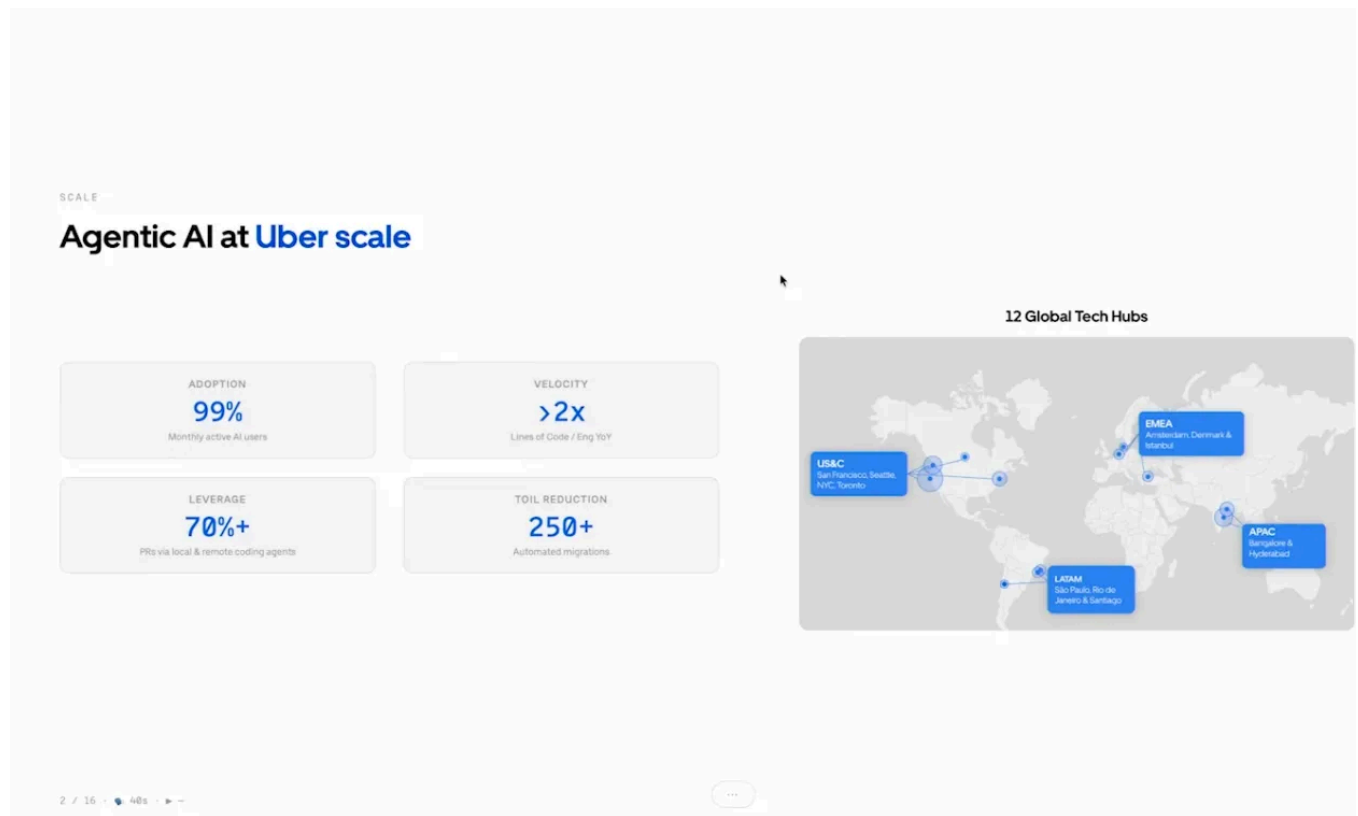
9



1



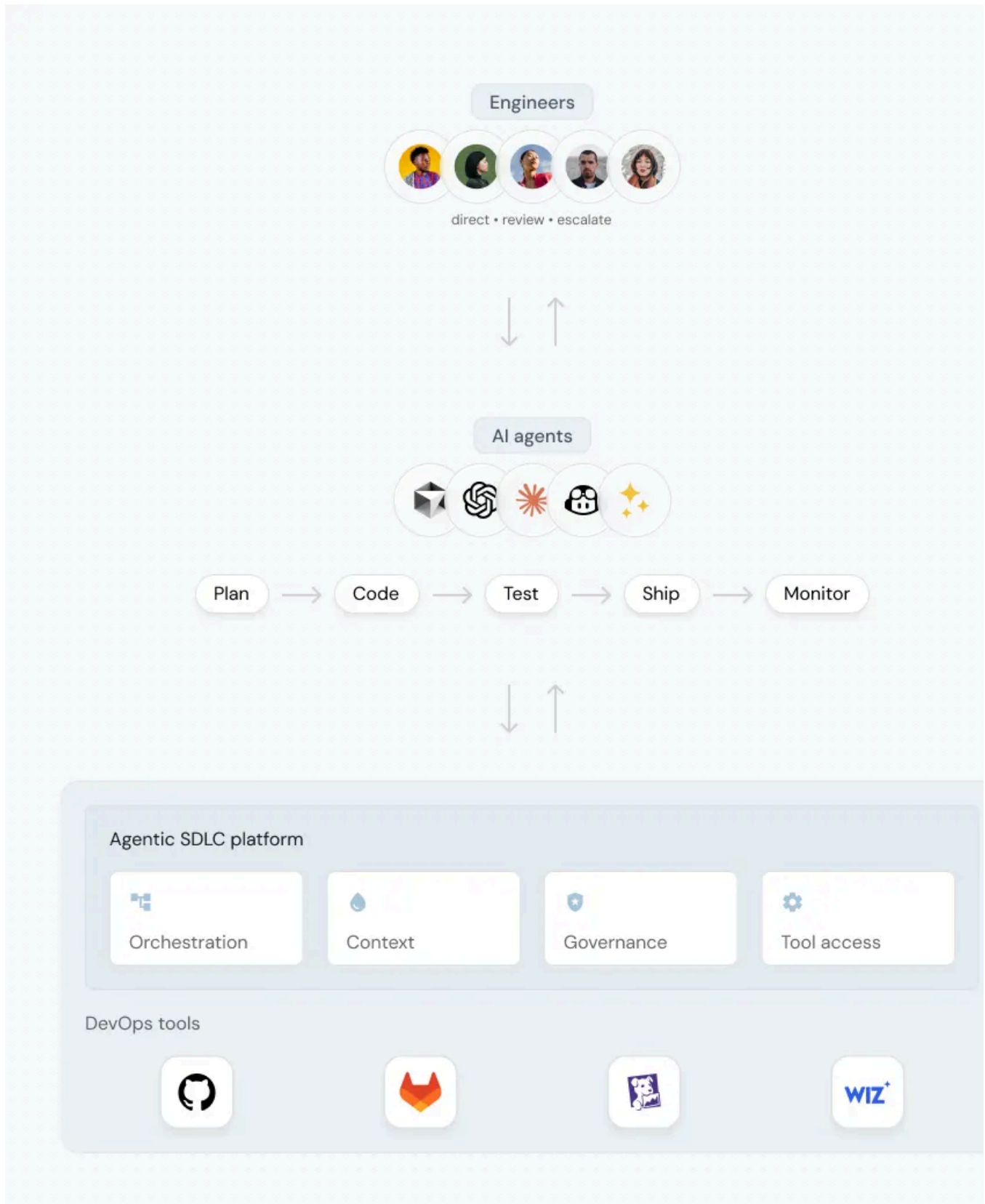
1



What is an AI Software factory in simple words?

An AI software factory is an operating model where your [software development lifecycle](#) runs as a production line, with agents doing the work at each stage and engineers designing and running the line.

The stages are the ones you already know. You plan the work, agents write code, engineers direct and review it, CI runs the tests, CI/CD ships it, and on-call keep healthy. What changes is who works each station: agents now stand beside your engineers, who move from writing every line to directing and approving it. You keep the same tools, and what changes is how the work moves through them.



How Uber built an AI software factory

Uber built an end to end [AI Software Factory](#). To get there they built an [agentic platform](#): it gives agents the pieces they need to work safely across a few thousand

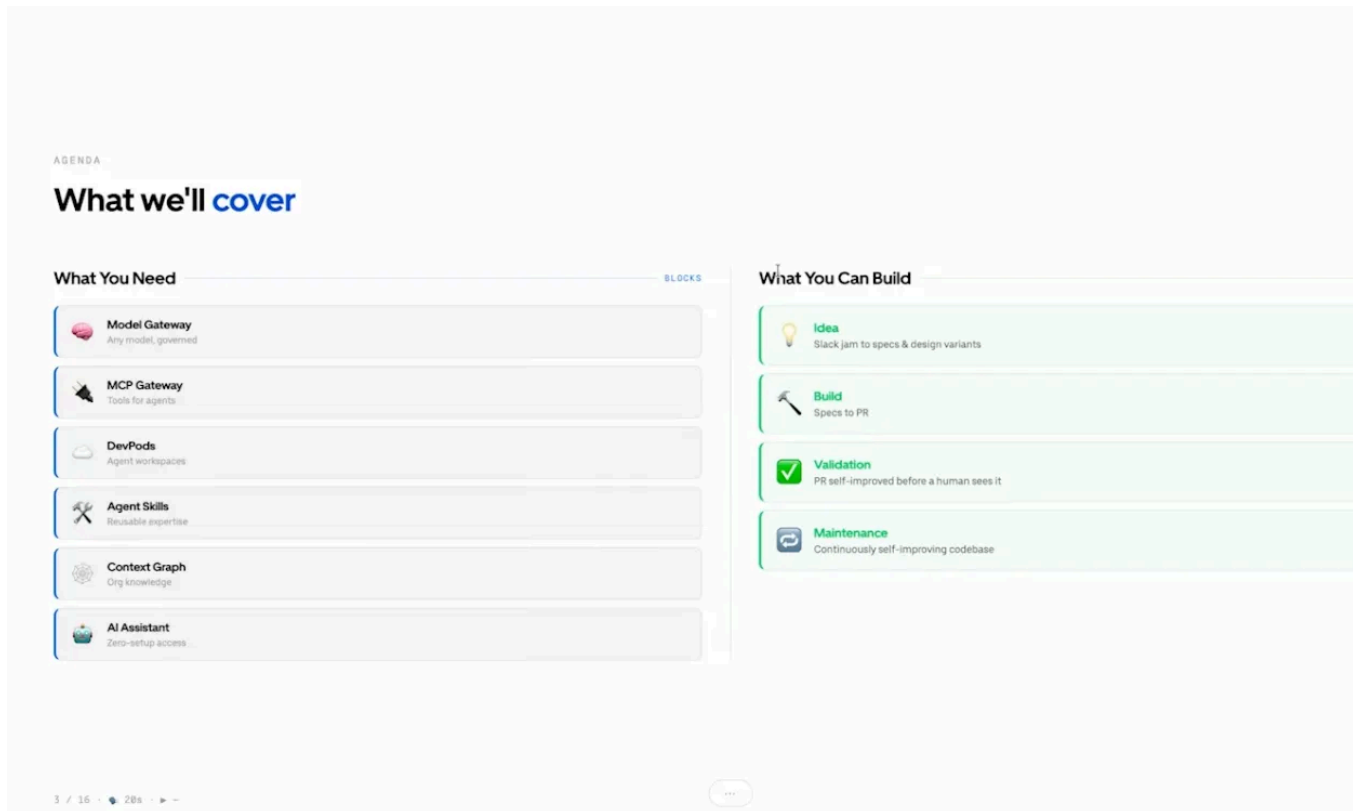
engineers: a [context graph](#) for organizational knowledge, an MCP gateway for to an LLM gateway for models, somewhere to run, and governance over all of it. On of that platform sit the workflows that turn agentic coding into shipped software carrying a feature from an idea to a merged PR and keeping it maintained with DevOps automation afterward. The workflows are the visible part, and they only because the platform underneath them is already there.

This piece walks through both layers, using Uber's own talk as the map. It answers four questions:

- What is an MCP gateway, and why does agentic coding at scale need one?
- What did Uber actually build to let AI agents work safely across a few thousand engineers?
- How do the spec-to-PR workflows and DevOps automation sit on top of that platform?
- Why does the platform, including the MCP gateway and the LLM gateway, have to come first?

The first half covers the platform, one piece at a time. The second half covers the workflows, and where each one reaches back into the platform to do its job.

Start with the platform, not the agents



The platform gives AI agents what they need to work across the software development lifecycle - the result is an [AI Software Factory](#). It does for agents roughly what an operating system does for applications. When you write an app, you do not manage memory or schedule CPU time yourself. The OS handles that, and you get on with your app. Agents need something similar underneath them, to handle [context](#), tools, resources, access, and safety, so the workflows on top can stay simple.

Uber's platform is made of six pieces. Most of them are infrastructure ideas you've already recognize, applied to agents rather than people.

Build one context graph instead of querying twenty systems

The screenshot displays the 'Context Graph' interface. On the left, a sidebar lists navigation items: 'Model Gateway', 'MCP Gateway', 'Dashboards', 'Agent Skills', 'Context Graph', and 'AI Incident'. The main content area is titled 'Context Graph' and includes a section 'WHAT WE NEEDED TO SOLVE' with three bullet points:

- Agents guess without relationships:** Agents can't traverse connections they can't see.
- Context is scattered across systems:** 30+ sources including incidents, ownership, code changes, feature flags.
- Fan-out burns tokens and time:** Assembling connected context takes many tool calls & increasing unpredictability.

The central part of the interface shows the 'AI Context Graph' as a complex network diagram. The diagram features a central node labeled 'Service' in a purple oval. Numerous other nodes are connected to it, including 'Engineer', 'Team', 'API Endpoint', 'Feature Flag', 'On-Call', 'Incident', 'Experiment', 'Deployment', 'Work Item', 'Documentation', 'Code Change', 'Repository', 'AI Tool', 'Chat Channel', 'Screen', 'Mobile App', 'Metrics', 'Pipeline', 'Alert', 'Event Stream', 'Dashboard', 'Supervisor', 'New', 'Design Doc', 'Jira Issue', and 'Bug'. The nodes are color-coded and connected by lines representing relationships.

At the bottom of the slide, the text reads: 'LEADERSHIP 1 • JUNE 30, 2026' and 'AI-Native Enterprises'.

Of the six pieces, this is the one worth understanding first.

Watching their own traces, Uber saw agents spending time and tokens just finding basic context. Where a service lives, what it depends on, who owns it, what patterns follow. That information was spread across 20 to 30 systems, and each one needed its own query. The result was added latency and unpredictable answers.

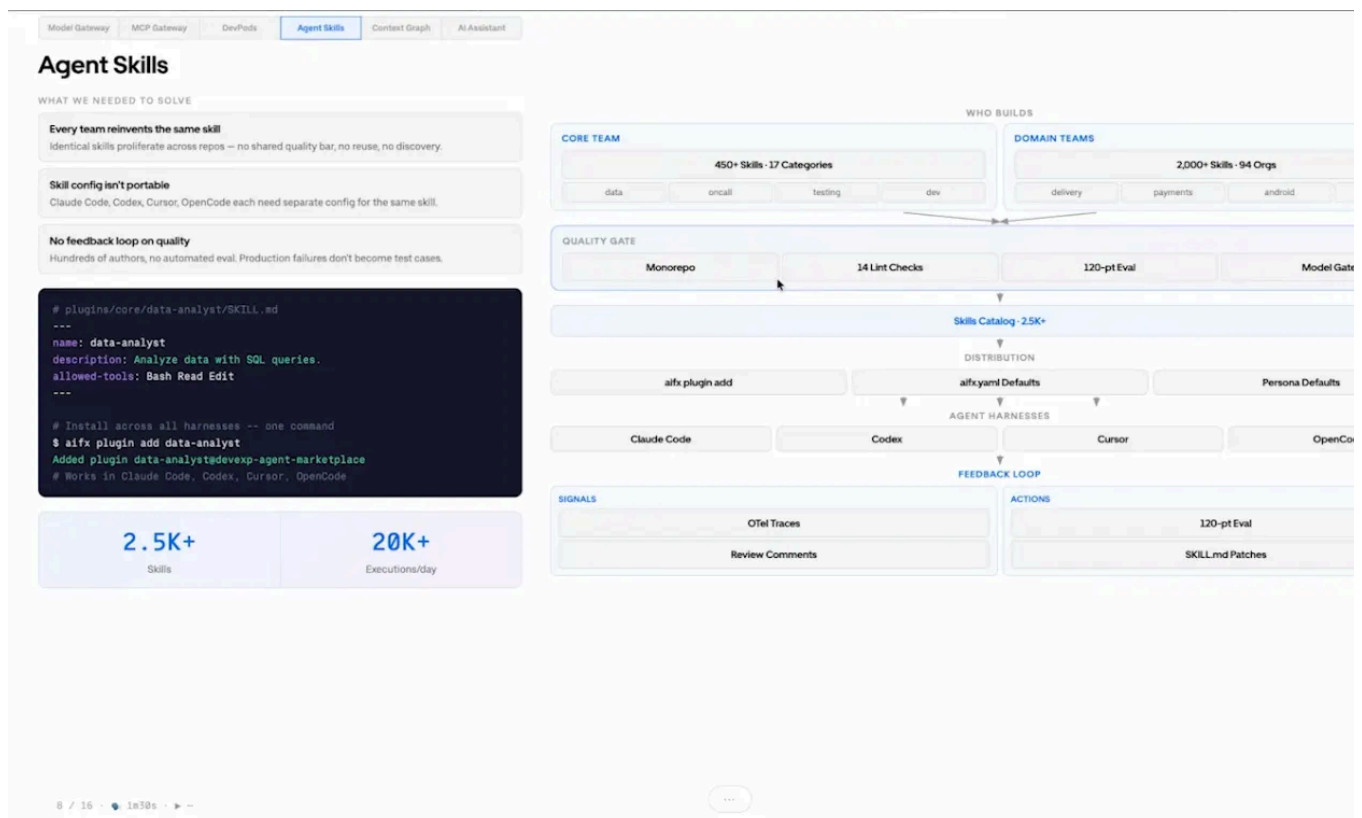
Their fix was to pull how the company actually runs into a single graph. It holds over a million entries across 150 kinds of nodes and edges. It connects how the mobile apps are built, the backend, and the data lake, along with design docs, Jira, incidents, and bugs, all linked together.

In other words, instead of the agent playing detective across thirty systems every time to work out what a service is and who owns it, they gave it one accurate, connected

map of the company to read. Think of it as a wiki that is always current and fully linked, rather than thirty stale pages you have to stitch together yourself.

Uber calls this a context graph. Platforms built for this pattern call the same idea [context lake](#), and most of the other pieces lean on it. Ask their graph “how many mobility trips in India are paid in cash” and it already knows the concepts, the tables and the entities needed to write the query. When they run the same task with the graph and without it, they see large drops in tokens, turns, and latency.

Curate skills in a registry, not across random repos



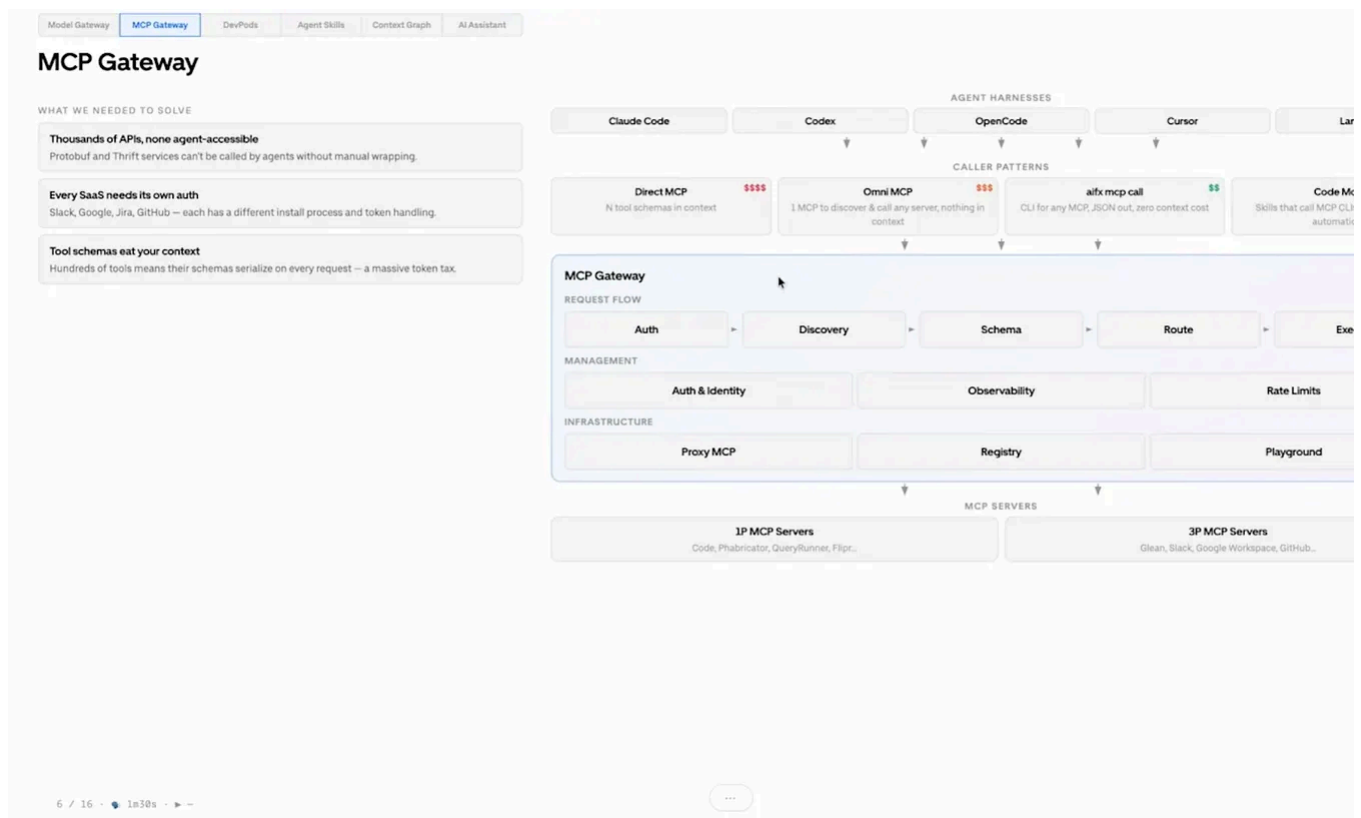
Once engineers can write skills for agents, they write a lot of them, and three people can show up fast. The same skill gets built five times by five people. Finding and configuring them is a hassle. And quality is all over the place.

In other words, a skill here is a reusable set of instructions that teaches an agent to do one task, like an internal library or a runbook, except written for an agent. In any library ecosystem with nobody curating it, it sprawls.

To handle that, Uber built a lifecycle around skills. Core and team-specific skills into one managed registry, now holding 2,500 of them, and everything passes automated lint and review so there is a quality floor.

Discovery and installation are simple. One command finds and installs any skill. based on what an engineer works on, the right defaults get installed automatically the agent usually has what it needs without anyone choosing. They also collect traces and run continuous evals, then feed the results back to whoever wrote the Across the fleet that comes to more than 20,000 skill runs a day.

Route every tool through one MCP gateway



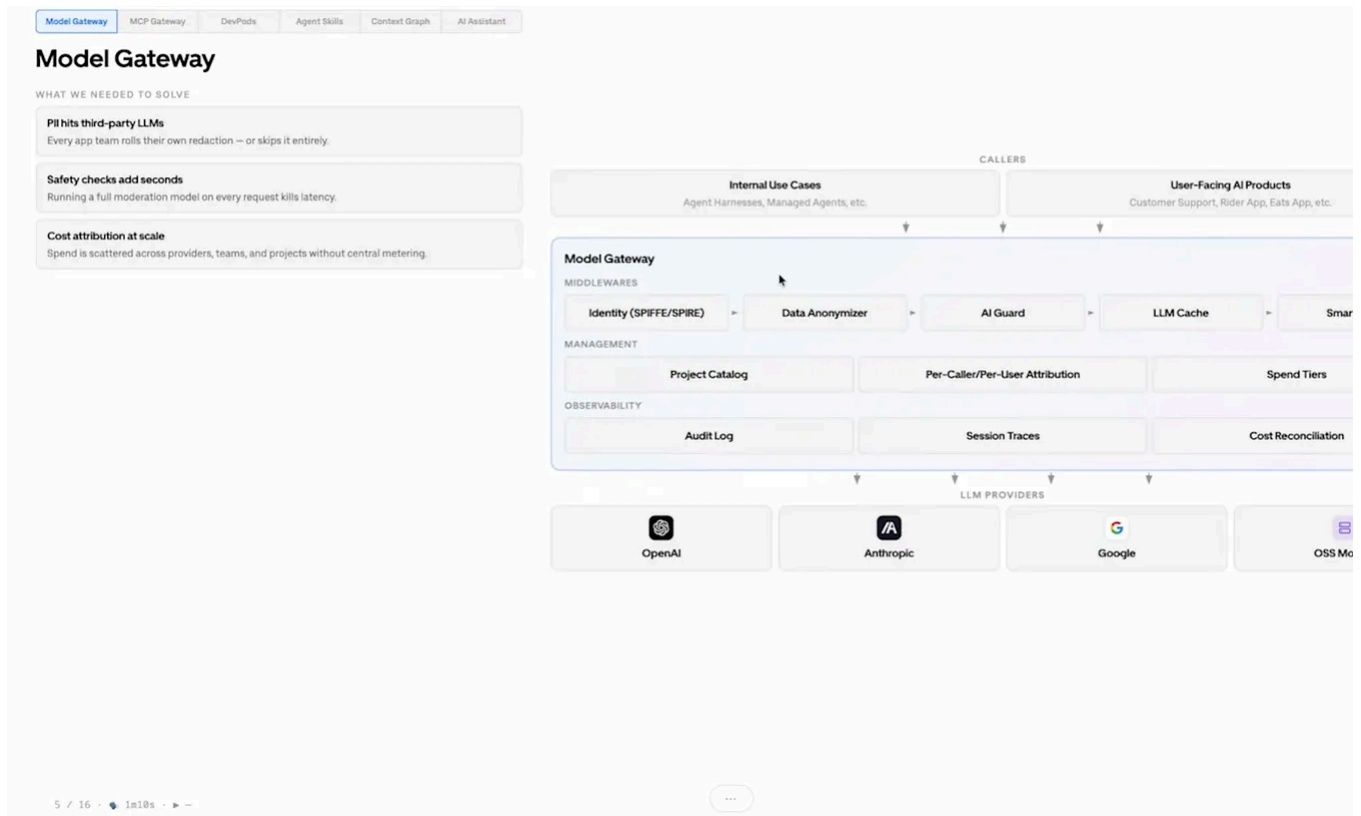
An MCP gateway is a single, governed entry point that connects AI agents to the and APIs they can call. An agent is only useful because of the tools it can reach, ; Uber started with thousands of internal APIs, none of them agent-ready, plus a p SaaS tools that each log in differently. Wiring these up created a new cost, becau every tool an agent can see eats context tokens before it does any real work.

In other words, an agent has a limited working memory, and every tool description hand it fills some of it up. Give it fifty tools and it has read a phone book before you have asked your question.

Their MCP gateway handles the setup problem first. An automated crawler turns internal API into an agent-ready tool with one config change, and SaaS tools like Google, Slack, and Jira go through the same gateway, which hosts them and handles the login handshake.

From there they cut the token cost in stages. First they replaced the pile of tools with a single tool that discovers and calls any of the others on demand, so the agent is not carrying all of them at once. Then they had responses come back in a compact format so the answers stop eating context. Most recently they added a mode where the agent writes small scripts on the fly for the heaviest cases. Together that work cut their token usage by more than 40% across the fleet, with over 1,000 tools now reachable this way. This is the piece a platform team should copy first if agents keep running out of context, because an MCP gateway is where tool sprawl either gets controlled or quietly breaks your agents.

Send every model call through one LLM gateway



An LLM gateway is a single, governed entry point that every model call passes through. If you let thousands of engineers call models directly, you lose control of three things at once. What data leaves your network. How much delay your safety checks add. Who is spending what. The LLM gateway is how Uber keeps hold of three, and it is the piece that makes the other five behave as a platform, because the point where every request is checked and accounted for.

Uber routes every model call, internal and external, through a single gateway that speaks the standard model APIs. Behind that one door sits a short chain of checks. Identity and auth come first. Then a step strips more than 20 kinds of PII, so not sensitive leaves the network. After that, an “AI guard” of five smaller models handles safety and policy.

Their hard rule is that all of this finishes in under 100 milliseconds, so the guard never becomes the reason a request is slow. Every call is also tagged to a project, a team, and a team, so spend and behavior can be traced in real time.

In other words, this is the AI agent governance layer. AI agent governance is the controls that keep what agents can do, see, and spend inside policy, and here it is the

the same shape as the API gateway already sitting in front of your services check auth and rate limits. Applied to every model call and given a strict latency budget means nobody bothers routing around it. Uber's [governance](#) layer now carries over a million model requests a day across 800-plus projects. Without something like it, agents at scale become a compliance and cost problem that tends to surface only when it is expensive.

Keep environments warm so agents start in seconds

The screenshot displays the DevPods platform interface, which is designed to manage and provision development environments for autonomous agents. The interface is organized into several sections:

- Model Gateway / MCP Gateway / DevPods / Agent Skills / Context Graph / AI Assistant**: Navigation tabs at the top.
- DevPods**: Main title for the platform.
- WHAT WE NEEDED TO SOLVE**: A list of challenges:
 - Agents can't run on laptops**: Autonomous agents need isolated, ephemeral envs.
 - Env setup kills agent speed**: Cloning a monorepo, installing deps, and indexing code takes minutes.
 - Language silos fragment execution**: Separate Go, Java, Web, iOS envs mean agents can't work across repos in one session.
- Code Snippets**:
 - Human: SSH + launch any agent: `ssh your-devpod-name`, `aix agent run claude` # or `codex`, `opencode`
 - Autonomous: ephemeral devpod via proto API: `DevpodCreate({ name: 'minion-49c6d1ac', node_type: 'devpod-agentio', flavor: 'go-large', region: 'us-or', state: STATE_RUNNING, })`
- Platform Components**:
 - MOBILE SIMULATION**: Includes **IOS Simulator**, **Android Emulator**, and **Visual Validation**.
 - AI DEVPOD**: Includes **AI-Optimized Image**, **One-Click Setup**, and **Non-Engineer Access**.
 - MEGA DEVPOD**: Includes **Go**, **Java**, **Web**, **Data**, **iOS**, **Android**, and a **Unified Environment** button.
 - INSTANT START**: Includes **Balloon Pods**, **Warm Pool**, **Snapshot Store**, and **Pre-indexed C**.
 - CORE RUNTIME**: Includes **K8s Pod**, **DinD Sidecar**, **SSD Volume**, and **Agent Runtime**.
- Metrics**:
 - 14K+** Running DevPods at peak
 - ~Secs** Startup time
- GKE REGIONS**: Includes **Oregon** (devpod-us-or), **Virginia** (devpod-us-va), **Netherlands** (devpod-nld), **São Paulo** (devpod-bra), and **M** (devpod-mex).

Agents need somewhere to run, and their requirements are not the same as a per An agent needs an isolated environment that comes up fast, runs for a long stretch can be spun up in bulk, and exists at every site.

Uber pre-provisions Kubernetes pods they call balloon pods. When an agent needs an environment, it grabs one that is already warm. The repos are snapshotted and the code search index is already built, so it is working within seconds.

In other words, instead of building a fresh environment every time an agent wakes up, which means cloning a huge repo and indexing it first, they keep a shelf of ready

machines with the code already loaded. It is like grabbing a rental car that is already fueled with the route set, rather than one you have to prepare yourself.

Give everyone one assistant across every surface

The screenshot displays the 'AI Assistant: Cortana' interface. At the top, navigation tabs include 'Model Gateway', 'MCP Gateway', 'DevPods', 'Agent Skills', 'Context Graph', and 'AI Assistant'. Below the title, a section titled 'WHAT WE NEEDED TO SOLVE' lists three challenges: 'Most users aren't in a terminal', 'Every team needs a different bot', and 'Some workflows need to run unattended'. To the left, a Slack message thread shows a user asking about mobility trip payments and Cortana providing a data-based answer. Below the chat, statistics show '300+ Personas' and '>20K Sessions/day'. On the right, a 'Cortana Platform' architecture diagram illustrates the layers: 'SURFACES' (Slack, Web UI, CLI, MCP), 'CAPABILITIES' (Personas, Automations, Memory), 'BACKEND' (Auth, Harness Router, Skill Router, Personalization), 'HARNESSES' (Claude Agent SDK, OpenCode SDK), and 'FOUNDATION' (Skills, MCPs, Context Graph, DevPods).

The last piece packages the other five for people to actually use. Their assistant, Cortana, has the context graph, the skills, and the tools plugged into it, and it shows up on Slack, the command line, and the web.

Anyone can ask it something, and it will check the context graph, run a skill, and code in any repo to answer. Teams can also personalize it with their own skills and prompts wired into a team channel, so it works like a member of that team. In one month they saw 300 team-specific versions and more than 20,000 sessions a day.

That covers the platform. On its own it does not ship anything. It is the layer the delivery workflows run on, and those workflows are the second half of the talk.

Then build the AI Software Factory workflows on top

With the platform in place, Uber can string its pieces into workflows that carry a feature from start to finish. Each of the workflows below leans on several of the platform pieces, which is the reason the platform has to exist before any of them

Take a feature from spec to prototype in one session

The screenshot displays the Uber AI Software Factory interface, specifically a workflow titled "From Specs to PRs". The interface is divided into several sections:

- Header:** Includes tabs for "Idea", "Build", "Validation", and "Maintenance".
- Section Header:** "From Specs to PRs" with a subtitle: "Minion orchestrates work across repos, validates, and delivers PRs."
- MINION - UBER'S CLOUD CODING AGENT:**
 - MODES:** "Interactive" (selected) and "Autonomous".
 - CAPABILITIES:** A grid of buttons including "Steer mid-task", "Cross-repo", "Review loop", "Your identity", "Persistent workspace", and "Every surface".
 - KEY LEARNINGS:** Three highlighted boxes:
 - "Engineers choose the task, not the model. Model routing is the platform's job."
 - "Cross-repository planning is the unlock. One task spans backend, mobile, and services. The agent understands how they connect."
 - "Toil workloads came first. End-to-end feature work required more. Making validation first-class on every PR was the unlock."
 - POWERED BY:** Buttons for "Model Gateway", "MCP Gateway", "Agent Skills", and "DevPods".
- NEW TASK:** A section on the right with a "World Cup 2026 - venue congestion pickup" task. It includes a description: "Two copy variants (A/B) behind a feature flag across all three repos." and a button labeled "World Cup Interstitial".
- HANDOFF FROM CORTANA TO MINION:** A section showing a checklist:
 - Product requirements:** ✓ Venue congestion interstitial · A/B copy
 - Engineering design:** ✓ go-code · ios-code · android-code
 - Design:** ✓ Figma: upfront interstitial mockup
- Summary:** "2 variants · feature flagged · across 3 repos"

Their walkthrough started with an idea kicked around in Slack. Someone tagged an assistant in, and things moved quickly from there.

Backed by the context graph, the assistant helped size the opportunity, then moved into a web session for deeper research. From there it generated two mockup variants for an A/B test, and reasoned about which existing screens and backend services team could reuse. Work that used to take weeks of alignment compressed into a 15-minute session and a prototype.

None of that runs without the platform. The research came out of the context graph, the mockups and the reasoning about existing code came from skills, and the whole thing ran through the assistant.

Hand off to an AI coding agent, but stop before CI

The prototype went to Minion, one of Uber's AI coding agents. Agentic coding is software development where an agent takes on a whole task, writing and changing code across a codebase with limited human steering, rather than only suggesting the next line. Minion is Uber's version of that, and it runs in one of those warm environments so it can change backend and frontend together.

One detail is worth calling out. Minion stops at a draft PR and does not push to CI. Uber found that pushing straight to CI was fine for small cleanup work but wasteful for real features, because it hammers shared CI before anyone has confirmed the feature even works. So they validate first.

Move your checks into the inner loop

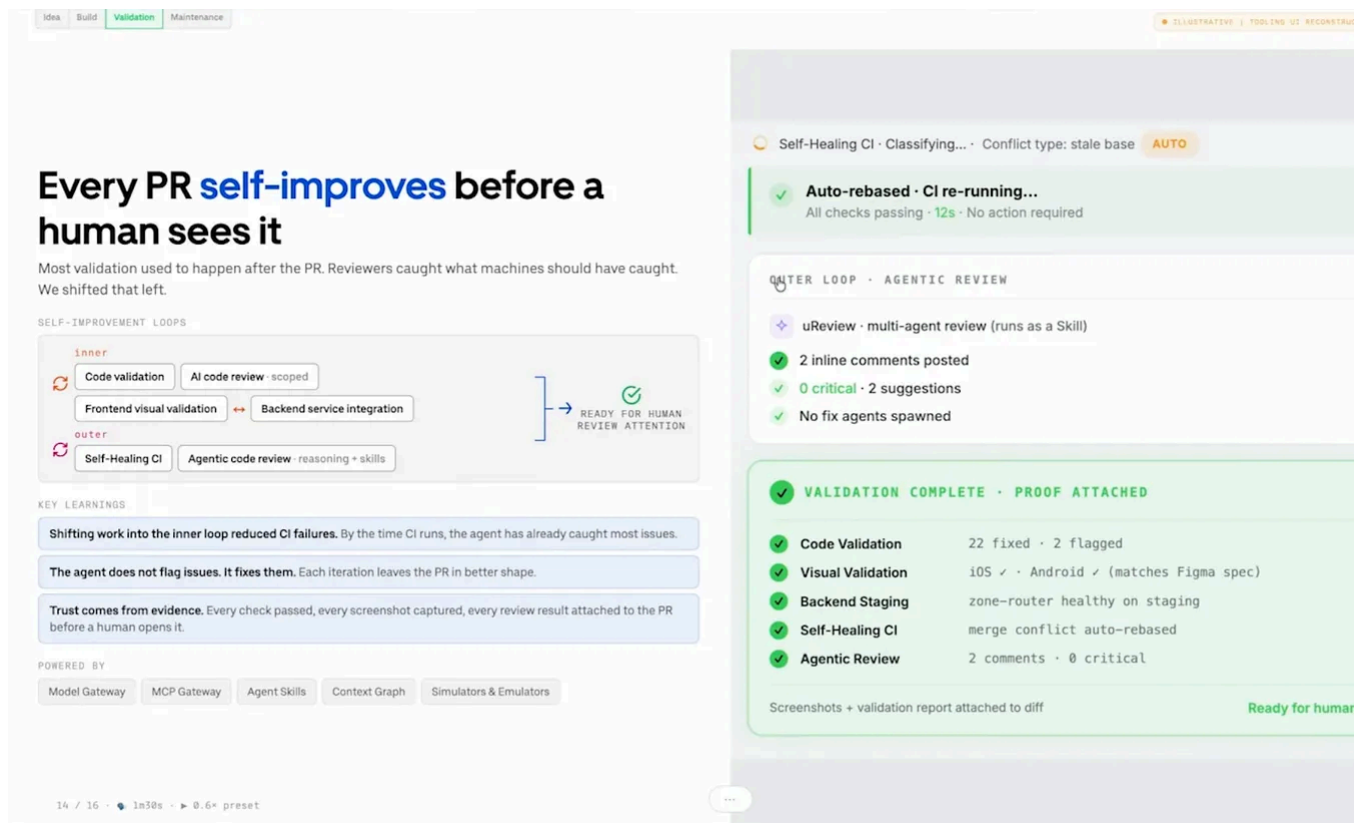
That validation happens in the inner loop, before CI. In other words, the inner loop is everything that runs on the agent's own machine before it pushes, and the outer loop is everything that runs after, in shared CI.

Uber moved several checks that used to live in CI back into the inner loop:

- Static analysis, the same lint-style checks as before, just run earlier.
- Visual validation, where a skill boots a simulator, takes a screenshot, and compares it against the design.
- Integration checks, which stand up the backend in staging so the front end and back end can be tested together.

Every one of those checks is a skill from the registry, running in a warm environment using the context graph to know what "correct" looks like. The idea is to have the platform do the pre-flight, so shared CI only sees work that is already likely to pass.

Let CI heal itself and show its work



Once the inner loop passes, the change reaches CI. There, self-healing CI fixes n failures on its own. This is DevOps automation aimed at agent-scale output: when agents open far more PRs, the pipeline has to repair routine breakages without a human in the loop. Code review is split the same way, with a smaller, faster model reviewing on the machine and a bigger, slower model doing a deeper review in C

Because a human is now reviewing a diff an AI coding agent wrote, the PR arrives with a table listing every check it already passed, screenshots included.

In other words, instead of asking a reviewer to trust a raw first draft, the PR shows work, the way you would trust a change more if you could see green checks and a run attached. The reviewer then spends the time on whether the change is a good rather than on catching basic mistakes.

Run maintenance as one managed loop

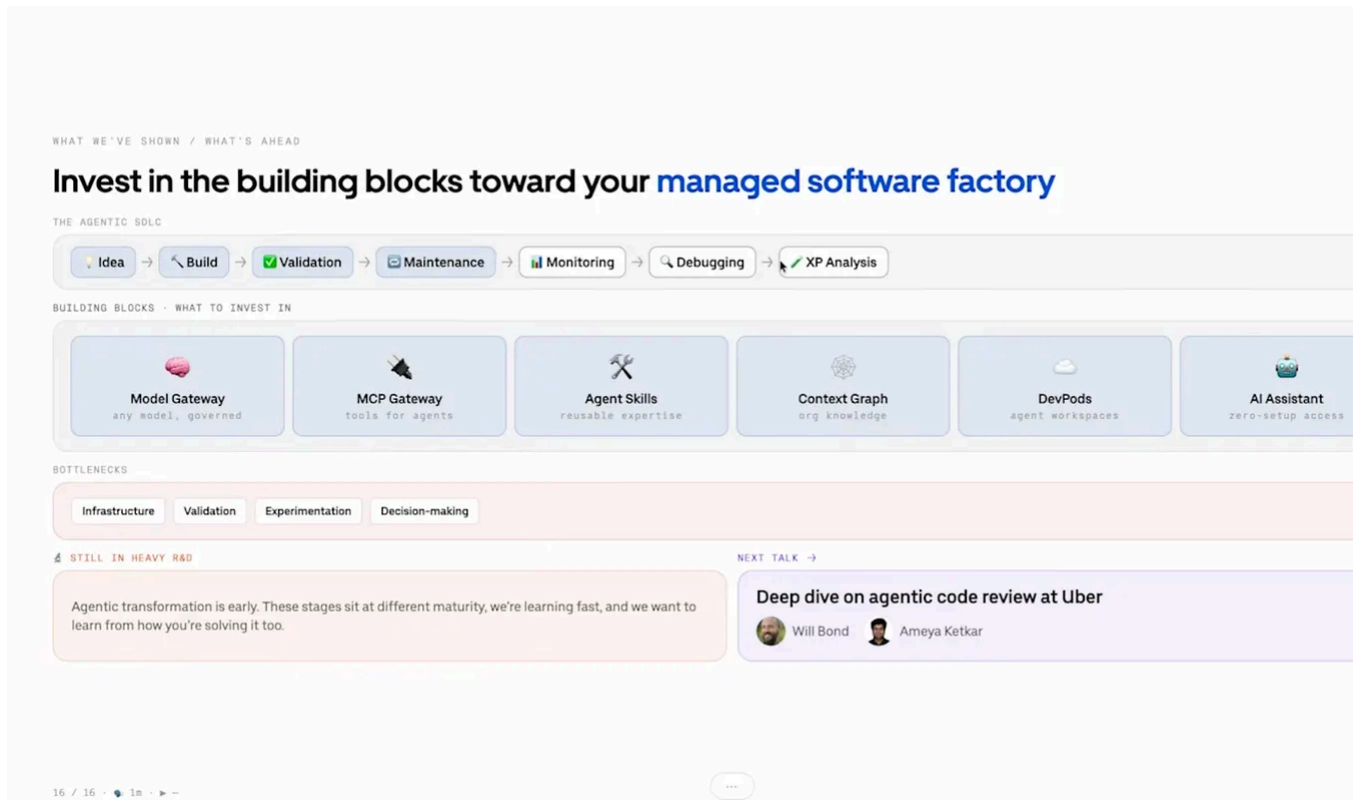
More code means more to maintain, so Uber treats maintenance as its own work and it is some of the clearest DevOps automation in the talk. You enroll a service maintenance skills, like one that cleans up a feature flag and removes the losing variant of an A/B test once it is decided.

The important property here is that the loop is managed centrally, which is AI as governance applied to recurring work. Rather than let thousands of loops run all the company where nobody can see or stop them, there is one place you go to set loop up. It runs on Sunday when CI has spare capacity, and it caps how many diffs on an engineer's Monday.

In other words, instead of every team quietly setting up its own cron job to have agent change things on a schedule, which is a good way to wake up to chaos nobody can trace, there is one controlled surface for all of it, with limits and a schedule you can reason about.

The loop also feeds itself. When those diffs get comments and either land or not, becomes useful data for improving the skill. And once a month, Uber mines its incident reviews for new maintenance skills to roll out across every service.

Build the platform first



The honest note Uber ended on is about where the constraint sits now. With this much code moving through the [AI Software Factory](#), the strain shows up in CI capacity, in how many experiments they can realistically run, and in decisions about what to actually build. As one of them put it, the question is no longer whether they can build something. They know they can. It is whether they should.

That kind of problem is only available to a team that has already built the platform. Every workflow leans on it:

- Spec-to-prototype needs the context graph.
- The validation loops need the skills registry and the warm environments.
- Every action needs the MCP gateway, and every model call needs the LLM gateway.
- The maintenance loop needs AI agent governance to stay bounded.

Without the platform underneath, the [AI Software Factory](#) is a set of scripts that the first time a repo moves or a model changes.

So if you are planning to let AI coding agents write a real share of your code, the platform is where to start. The workflows are the part you will want to show people but they sit on top of a context graph, an MCP gateway, an LLM gateway, environments, and a way for people to reach all of it. Uber built that groundwork and the numbers everyone is quoting are what it produced.

Where [Port](#) fits

Uber built all six pieces in-house, which very few teams have the scale to do, and closing that gap is what Port is for. Port gives you the same foundation without building it from scratch: a [context lake](#), an MCP hub, agent and skills management workflow orchestration, and [governance](#) across the SDLC. On top of that foundation teams run solutions like [autonomous ticket resolution](#), [self-healing incidents](#), an [engineering intelligence](#). Companies including GitHub, dLocal, and PwC already use it to move from manual to [agentic engineering](#), and you can read how in their [customer stories](#).

Frequently asked questions

What is an MCP gateway?

An MCP gateway is a single, governed entry point that connects AI agents to the tools and APIs they can call. It standardizes authentication, hosts internal and third-party tools as MCP servers, and reduces the context tokens each tool consumes. Uber's gateway made over 1,000 tools reachable and cut token usage by more than 40%.

What is an LLM gateway?

An LLM gateway is a single, governed entry point for every model call an organization makes. It handles authentication, redacts sensitive data, and applies safety checks then attributes spend to a team, usually within a strict latency budget so it doesn't slow requests down. Uber's LLM gateway carries over 100 million model requests a day.

What is agentic coding?

Agentic coding is software development where AI agents take on whole tasks, write, changing, and validating code across a codebase with limited human steering, rather than only suggesting the next line. It depends on a platform that gives the agent organizational context, tools, and governed model access.

What is DevOps automation?

DevOps automation is the practice of handing repetitive software delivery and operations work to automated systems instead of people, including CI fixes, migrations, and routine cleanup. At Uber it runs at agent scale, covering more than 250 automated migrations and self-healing CI that repairs pipeline failures on its

What is AI agent governance?

AI agent governance is the set of controls that keep what AI agents can do, see, and spend inside policy. In practice it includes routing model calls through a governed LLM gateway, attributing every request to a team, redacting sensitive data, and bounding autonomous work so it stays observable.

Credit: This writeup is based on a talk given by two Uber engineers, Uday Kiran Medisetty and Adam Huda, who walked through the platform and then the end-to-end feature flow in detail. The numbers, the architecture, and the design calls are all This blog summarizes what they shared so more teams can learn from it.



9 Likes • 1 Restack

Discussion about this post

Comments Restacks



Write a comment...



Alessandro Pieraccini 30 Aug

♥ Liked by Zohar Einy

Thank you for sharing. I really enjoyed reading your article and congratulations on the simplicity with which you explained concepts that are actually not simple at all.

The element that caught my attention the most was the work done on the MCP gateway. With hundreds or thousands of tools, the problem concerns access, governance, and how much context is consumed before the agent even starts working.

When we make tools available on demand, we reduce token usage and probably also the complexity of tool selection. It is a great example of how much the reliability of an agentic system depends on the architecture built around the model, not only on the model itself, which, even if well designed, would still carry structural limitations and inefficiencies of its own.

♥ LIKE (1) 💬 REPLY

